BUSN 5000 Project Guide

Chris Cornwell and Abbi Cormier

May 18, 2026

Table of contents

Preface

Welcome to the **BUSN 5000 Project Guide** — the canonical reference for the Pre-project, Project Progress Check, and Final Project for BUSN 5000 (Intro to Data Science for Business and Economics) at the University of Georgia.

This guide covers:

- How to set up your computer and your project folder
- What the pre-project is and how to complete it
- What the main project is and how to complete it
- What the progress check is and how it's graded
- How to submit your work
- How to recognize and recover from common errors

The guide is the single source of truth. Announcements on eLC remind you of upcoming deadlines and point back here for the rules.

How to use this guide

Read the chapters in order the first time through. After that, treat it as a reference — jump to whatever section you need. Each chapter has its own table of contents in the sidebar.

If you find a section that's wrong, unclear, or out of date, please flag it via the "report an issue" link at the top of any page.

Acknowledgments

The course materials in this guide are by **Chris Cornwell** and **Abbi Cormier**, with substantial contributions from the BUSN 5000 teaching team across many semesters.

1Setup

This chapter walks you through getting your computer ready to do BUSN 5000 project work. By the end of it, you should have:

- R and RStudio installed
- All required R packages installed
- A `Project/` folder on your computer with the correct subfolder structure
- Every file from the Pre-project file pack sorted into the right place
- An `.Rproj` file that opens RStudio into the project
- A green report from the setup-check script

Plan to spend 30–45 minutes the first time through.

> **!** Don't skip this
>
> Almost every "my project won't knit" email we get traces back to something here. If setup is right, the rest of the project is much easier. If setup is wrong, every other step compounds the problem.

1.1 Two separate folders to keep straight

You'll have **two** different folders for BUSN 5000:

1. **Your BUSN 5000 folder** — a general place for homework assignments and class materials. The Getting Started Day slides cover this. *This chapter is not about that folder.*
2. **Your Project folder** — a self-contained R project that holds the Pre-project, Project Progress Check, and Final Project. Call this folder `Project`. **This chapter is about this one.**

Keep them separate. The Project folder is its own R project with its own subfolder structure; the BUSN 5000 HW folder is a plain folder.

1.2 Install R and RStudio

If you haven't already, install both. Follow the instructions for your operating system. Install R before RStudio.

1.2.1 Windows

Step 1: Install R

1. Go to the CRAN R Project website.
2. Click on "Download R for Windows".

3. Click on "base" and then select the link labeled "Download R x.x.x for Windows" (where x.x.x is the latest version).
4. Open the downloaded `.exe` file and follow the installation prompts, accepting the defaults.

Step 2: Install RStudio

1. Visit the RStudio download page.
2. Click the link to download the Windows installer.
3. Open the downloaded `.exe` file and complete the installation using default settings.

Step 3: Verify the installation

1. Launch RStudio from the Start menu or desktop shortcut.
2. In the Console pane, type `print("Hello, world!")` and press Enter.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.2.2 macOS

First, identify your Mac's chip. Newer Macs use Apple Silicon (M1, M2, M3, M4, etc.); older ones use Intel processors. You need to know which you have to pick the right R installer.

1. Click the Apple menu (top-left corner) → **About This Mac**.
2. Look at the **Chip** (Apple Silicon) or **Processor** (Intel) line:
   - If it says "Apple M1", "M2", "M3", "M4", or any Apple-labeled chip, you have **Apple Silicon**. You want the `arm64` installer.
   - If it says "Intel Core …", you have an **Intel Mac**. You want the `x86_64` installer.

Step 1: Install R

1. Visit the CRAN R Project website.
2. Click on "Download R for macOS".
3. Select the latest version matching your chip — either `R-x.x.x-arm64.pkg` (Apple Silicon) or `R-x.x.x-x86_64.pkg` (Intel).
4. Open the downloaded `.pkg` file and follow the installation prompts.

Step 2: Install RStudio

1. Go to the RStudio download page.
2. Click the link to download the macOS version of RStudio.
3. Open the downloaded `.dmg` file and drag the RStudio icon to your `Applications` folder.

Step 3: Verify the installation

1. Open RStudio from your `Applications` folder.
2. In the Console pane, type `print("Hello, world!")` and press Return.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.3 Install the required R packages

Open RStudio. In the Console pane (bottom-left), paste this single line and press Enter:

```
install.packages(c("here", "tidyverse", "modelsummary", "ggpmisc", "car", "ggrepel", "kableExt
```

This installs all seven packages the Pre-project, Progress Check, and Final Project will need. Installation takes a few minutes the first time — let it finish completely.

> **Tip**
>
> You only install packages **once per machine**. After that, you load them with `library()` in your `.Rmd` (which the templates already handle). Don't put `install.packages()` calls inside a `.Rmd` — that would re-install every time you knit.

1.4 Create the `Project` folder and its subfolders

1. Decide where on your computer your `Project` folder will live. Anywhere is fine **except your Downloads folder** (see Common mistakes below).
2. Create a folder there called exactly `Project`.
3. Inside `Project`, create these four subfolders:
   - `data/`
   - `out/`
   - `r/`
   - `css/`

So your skeleton looks like:

```
Project/
  data/
  out/
  r/
  css/
```

You'll put files into these subfolders in the next step. The `.Rmd` files themselves will live in the `Project/` root, **not** in any of the subfolders.

1.5 Sort the Pre-project file pack into the right places

When you download the Pre-project file pack from eLC, you'll get a flat bucket of 8 files. Your job is to figure out where each one belongs in your `Project/` directory. This isn't busywork — knowing which file goes where is itself part of what we want you to learn.

Here's the destination for each file in the pack:

File	Goes in
`pre_project.Rmd`	`Project/` (root)
`pre_project_knitted_template.html`	`Project/` (root)

File	Goes in
Pre-project Instructions (PDF)	`Project/` (root)
`LF.R`	`Project/r/`
`project_slidedeck.css`	`Project/css/`
`pppub24.csv`	`Project/data/`
`hhpub24.csv`	`Project/data/`
`check_setup_preproject.R`	`Project/r/`

After sorting, your folder should look like:

```
Project/
   pre_project.Rmd
   pre_project_knitted_template.html
   Pre-project Instructions.pdf
   data/
       pppub24.csv
       hhpub24.csv
   out/
    (empty for now - LF.R will write LF.csv here)
   r/
       LF.R
       check_setup_preproject.R
   css/
        project_slidedeck.css
```

1.6 Create the `.Rproj` file

The `.Rproj` file is what tells RStudio "this folder is an R project." It anchors all the relative paths the templates use.

To create one:

1. Open RStudio.
2. **File → New Project**.
3. Choose **Existing Directory**.
4. Browse to your `Project/` folder. Click **Create Project**.
5. RStudio re-opens with `Project` loaded as the active project.

You'll see `Project.Rproj` appear in the `Project/` root. From now on, **always open RStudio by double-clicking that `.Rproj` file**. That guarantees your working directory and paths are correct.

1.7 Run the setup check

In the RStudio Console, with `Project` loaded as the active project, type:

```
source("r/check_setup_preproject.R")
```

The script verifies every required file is in its correct subfolder. You'll see one line per file: either (good) or with a specific instruction telling you what to fix.

> **Note**
>
> If you see any , fix the listed problems and re-run the script. Don't move past Setup until every line is a .

1.8 Common setup mistakes

In our experience, these are the three setup problems we see most often. Each one is preventable with a few seconds of attention.

1.8.1 Mistake 1: `project_slidedeck.css` in the wrong folder

The CSS file styles your slide deck. If it's anywhere but `Project/css/project_slidedeck.css`, your knitted slides will lose all of their formatting and the submission will be rejected. The setup-check script will catch this, but it's a simple thing to get right on the first pass.

1.8.2 Mistake 2: Working from your Downloads folder

If you double-click `pre_project.Rmd` directly from your Downloads folder, RStudio's working directory is Downloads, not your `Project/` folder. The `here()` paths in the `.Rmd` will resolve to the wrong place and you'll get cryptic "file not found" errors.

Always move the files into your `Project/` folder before opening them. Your Downloads folder is a transit area, not a workspace.

1.8.3 Mistake 3: Knitting before running the R script

The `.Rmd` reads output that the `LF.R` script (and later `cpsmar_e.R`) produces. If you haven't run the script yet, the output file doesn't exist, and the `.Rmd` errors out the moment it tries to read `out/LF.csv`.

Workflow order for each stage:

1. Open the `.Rproj`.
2. Run the relevant R script (`LF.R` for pre-project, `cpsmar_e.R` for main project).
3. *Then* open the `.Rmd` and knit it.

The setup-check script can confirm files exist, but only running the script confirms it actually produced the output the `.Rmd` needs.

1.9 What happens next

You're set up. From here:

- For the **Pre-project**, continue to Chapter 3.
- For the **Main Project** (later in the semester), you'll add more files to the same `Project/` folder — see Chapter 4.
- For the **Project Progress Check**, you continue working on the same main project files — see Chapter 5.

Your `Project/` directory carries through the entire semester. You won't need to re-do Setup unless you're working from a new machine.

2Pre-project

This chapter covers the BUSN 5000 Pre-project Exercise.

2.1 Outline (to be written)

- Purpose of the pre-project
- What you'll produce (a 3-slide deck demonstrating setup)
- Step-by-step walkthrough
- Submission rubric (all-or-nothing 5 points; common formatting errors)
- Filename convention
- Common mistakes and how to avoid them

> **Note**
>
> This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

3Main Project

This chapter is the slide-by-slide guide to the BUSN 5000 Project: **Exploring Pay Differences between Women and Men**.

3.1 Outline (to be written)

- Project overview and learning goals
- Data: March 2024 CPS ASEC
- The 35-slide deck structure
- Slide-by-slide instructions (migrated from the current Instructions HTML)
- Point allocations per slide
- Line limits and write-up rules

> **Note**
>
> This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

4Progress Check

The Project Progress Check is an **optional bonus** opportunity that gives you feedback on your project halfway through and rewards progress.

4.1 Outline (to be written)

- What the progress check is and isn't
- Which slides are graded (Title, Academic Honesty, Distribution of earnings by gender, Wages and hours differences, Career log wage profiles, Evolution of the gender wage gap)
- Scoring: 5 points (clean), 3 points (formatting clean / content error), 0 (formatting error)
- How to submit
- How feedback is provided

> **Note**
>
> This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

5Submission

This chapter covers how to submit your work to Gradescope without losing points to formatting.

5.1 Outline (to be written)

- Filename convention (`sectiontime_lastname_firstname_*.pdf`)
- PDF format (landscape, color, formatting preserved)
- The HTML → PDF workflow (not knit-to-PDF)
- Browser-specific save-as-PDF tips
- What "100% compliant with the knitted template" means in practice
- Deadlines and late-submission policy

> **Note**
>
> This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

6Common Errors and FAQ

If something is broken, look here first.

6.1 Outline (to be written)

- The "did setup work?" diagnostic (`check_setup.R`)
- Common error catalog:
  - CSS in the wrong folder
  - `eval=FALSE` left on a completed chunk
  - YAML edited
  - Wrong working directory
  - Missing packages
  - Knit-to-PDF instead of HTML → PDF
  - Incorrect filename on Gradescope
- Communication policy (no content review by email; one thread per issue; no same-issue cross-emails)
- How to ask for help productively

> **Note**
>
> This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

7R and AI Resources

A curated list of resources for learning and using R, and for working with AI as part of your data-science practice. Nothing here is required reading — but each item below has earned its place. Skim it now, bookmark it, and return as you go.

7.1 Resources for learning and using R

7.1.1 IDEs for R

RStudio remains the most widely used IDE for R, and, like R, it is open-source. Its "Help" tab is a gateway to a wide range of R and RStudio resources. However, Posit (the company behind RStudio) has released Positron, a next-generation IDE built for data science with both R and Python. Positron combines the familiar feel of RStudio with the extensibility of VS Code, and includes built-in AI assistance via Positron Assistant. If you're comfortable with RStudio, stick with it; if you're curious about a more modern interface or plan to work across R and Python, Positron is worth exploring.

7.1.2 Learning R

Norm Matloff, a computer scientist and statistician at UC-Davis, has a nice, fast introduction to (base) R. He also has interesting takes on base R vs the Tidyverse and the R vs Python debate in data science. For a more in-depth treatment, consult the widely used R for Data Science by Wickham and Grolemund. They will guide you through the Tidyverse. Tidy Data Tutor helps you visualize your data analysis pipelines. James Scott has a great introduction to R that more clearly maps onto BUSN 5000.

You will find a list of econometrics packages for R at the Comprehensive R Archive Network (CRAN). You might prefer the causal inference task view or machine learning task view.

Learning R Markdown or Quarto should go hand-in-hand with learning R. For R Markdown, get started here and keep a cheatsheet handy. You can get started with Quarto here.

7.1.3 Workflow

At least as important as learning R is understanding basic workflow principles. R-bloggers has a beginner's guide using RStudio Projects. You can find some of the same principles in chapter 2 of Kieran Healy's Data Visualization book (also linked on the course schedule). Matt Gentzkow and Jesse Shapiro provide a fuller take on workflow principles from the perspective of academic social scientists (also posted on eLC). They even provide a manual for doing as they do.

For a perspective specifically on coding practices in data science education, Pruim, Gîrjău, and Horton (2023) make the case in "Fostering Better Coding Practices for Data Scientists" (*Harvard Data Science Review*) that good coding habits matter even for beginners. Their argument: it's far easier to learn good practices alongside R than to unlearn bad habits later. They organize their guidance into a practical top-10 list. Worth reading early in your data-science career.

Suffice it to say that Healy's book is a great introduction to the wonders of ggplot, which is not R, but indispensable to analysis done in R.

7.2 AI resources

7.2.1 Working with AI

My take is that you should view AI chatbots, like ChatGPT, Claude, and Gemini, as exceedingly productive collaborators with whom you should learn to work, much like you would with a human. Unfortunately, there are no secret prompting strategies or special step-by-step manuals (also like with a human). For some practical guidance along these lines, I recommend that you follow Ethan Mollick's Substack. If you want to swim in the deeper end of the pool, check out the Substack, Don't Worry About the Vase.

7.2.2 AI for coding

One place gen AI really helps is in coding. You can get coding help from chat interfaces, but you may prefer a more integrated experience. Several options exist:

- Positron Assistant: If you use Positron, this built-in AI assistant understands your R environment, loaded data, and console history. It integrates GitHub Copilot for code completions and Claude for chat and agent mode.
- GitHub Copilot: Works in RStudio, VS Code, and other IDEs. Here is how to get free access as a student. Here are instructions for RStudio integration.
- Claude Code: A command-line tool that can read and modify files in your project directory, run commands, and understand your codebase context. Simon Couch has written useful posts on using Claude Code for R package development.
- Cursor: An AI-first code editor built on VS Code.

Alternatively, you may find you can get by just vibe coding with a chatbot — describing what you want and iterating on the results.

7.2.3 Learning more about AI

To learn more about AI, including a comparison of different models, check out Generative AI for Beginners, a free course offered by Microsoft Cloud Advocates. OpenAI and Anthropic have their own AI academies when you want to step it up.